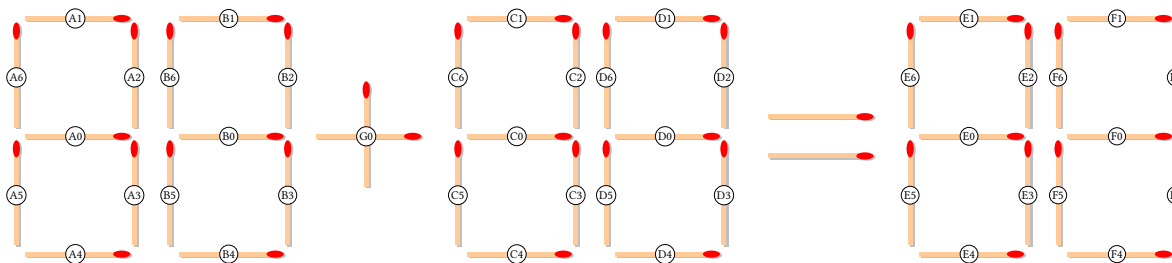


Γρίφοι με Σπίρτα

Οι γρίφοι με σπίρτα μας ζητούν να φτιάξουμε μια εξίσωση με σπίρτα, με έναν περιορισμένο αριθμό κινήσεων. Στον γρίφο από πάνω πρέπει να φτιάξουμε μια εξίσωση μετακινώντας ένα ή δύο σπίρτα. Μπορείτε να τη λύσετε; (Υπόδειξη: Υπάρχουν πέντε λύσεις).

Εμείς εδώ θέλουμε να κάνουμε τον υπολογιστή να λύνει τέτοιους γρίφους. Αν και μερικές φορές οι γρίφοι αυτοί απαιτούν δημιουργική και παράπλευρη σκέψη, θα δούμε ότι η χρήση της λογικής μπορεί να μας δώσει λύσεις. Δεν θα μπορέσουμε να λύσουμε κάθε δυνατό γρίφο με σπίρτα, αλλά θα δούμε ότι θα μπορέσουμε να λύσουμε πολλούς, χρησιμοποιώντας αλγοριθμικό τρόπο σκέψης.

Για να ξεκινήσουμε, θα πρέπει να αποφασίσουμε πώς θα περιγράψουμε τους γρίφους. Θα ασχοληθούμε με γρίφους που περιέχουν μέχρι τρεις διψήφιους αριθμούς, δύο από τους οποίους θα μπορούμε να προσθέσουμε ή να αφαιρέσουμε για να φτιάξουμε τον τρίτο. Θα αναπαράστούμε κάθε αριθμό με ένα κεφαλαίο λατινικό γράμμα, και τα σπίρτα που τον απαρτίζουν με αριθμούς. Κάθε αριθμός μπορεί να χρησιμοποιήσει μέχρι επτά σπίρτα. Το σπίρτο στη μέση οριζόντια θέση θα είναι το μηδέν· τα υπόλοιπα γύρω του θα αριθμούνται με τη φορά των δεικτών του ρολογιού, ξεκινώντας από την κορυφή. Ο τελεστής (συν ή πλην) επίσης θα αναπαρίσταται με ένα γράμμα και τον αριθμό μηδέν, όπως φαίνεται στην Εικόνα 1.



Εικόνα 1: Αναπαράσταση σπίρτων.

Ένας άλλος τρόπος να δούμε τους αριθμούς είναι όπως αυτοί σε παλιές ψηφιακές οθόνες, που χρησιμοποιούσαν επτά τμήματα για κάθε ψηφίο, όπως αυτό: $\exists: 14$. Όπως και να χει, θα συμβολίζουμε το σύνολο των σπίρτων, ή των τμημάτων, που χρησιμοποιούνται σε κάθε ψηφίο d με S_d . Τότε τα ψηφία αναπαρίστανται όπως στον παρακάτω πίνακα.

d	S_d
$\emptyset$	$\{1, 2, 3, 4, 5, 6\}$
1	$\{2, 3\}$
2	$\{0, 1, 2, 4, 5\}$
3	$\{0, 1, 2, 3, 4\}$
4	$\{0, 2, 3, 6\}$
5	$\{0, 1, 3, 4, 6\}$
6	$\{0, 1, 3, 4, 5, 6\}$
7	$\{1, 2, 3, 6\}$
8	$\{0, 1, 2, 3, 4, 5, 6\}$
9	$\{0, 1, 2, 3, 4, 6\}$

Το πρώτο πράγμα που πρέπει να κάνουμε είναι να χειριστούμε τον τελεστή. Υπάρχουν δύο εναλλακτικές. Μπορεί να είναι είτε $+$ είτε $-$, και θα τον δοκιμάσουμε με αυτή τη σειρά. Αναλόγως με τον αρχικό τελεστή, μπορεί να πρέπει να προσθέσουμε ένα σπύρτο (αλλάζοντας το $-$ σε $+$) ή να αφαιρέσουμε ένα σπύρτο (αλλάζοντας το $+$ σε $-$). Θα σημειώνουμε τι κάναμε σχετικά με $o_d \in \{0, 1\}$ και $o_r \in \{0, 1\}$ αντίστοιχα. Η διαφορά $o_d = o_r - o_a$ δείχνει τι μπορεί να μεταφερθεί μεταξύ του τελεστή και των αριθμών. Ξέροντας αυτό, μπορούμε να προχωρήσουμε με τους αριθμούς. Θα επεξεργαστούμε τους αριθμούς αναδρομικά, πηγαίνοντας από τα αριστερά προς τα δεξιά. Ο αλγόριθμος με τον οποίο θα επεξεργαστούμε τους αριθμούς σε κάθε θέση (slot) της εξίσωσης είναι στην ουσία κατά βάθος αναζήτηση, και αποτυπώνεται ως εξής.

Algorithm 1: Recursive Slot Processing

```

DoSlot( $i, ns$ )
  Input:  $i$ , the current slot
            $ns$ , the number of slots

  1  if  $i = ns$  then
  2    if we have a solution then
  3      record the solution
  4    return
  5  foreach candidate digit  $t_d$  in the current slot do
  6    if not impossible to find a solution then
  7      add  $t_d$  to the solution
  8      DoSlot( $i + 1, ns$ )

```

Για να γίνει λειτουργικός ο αλγόριθμος, πρέπει να αναλύσουμε πώς θα ελέγχουμε αν έχουμε βρει μια λύση όταν φτάσουμε στο ψηφίο στην τελευταία θέση (γραμμή 2), πώς θα εξερευνούμε τα υποψήφια ψηφία σε κάθε θέση (γραμμή 5), και πώς μπορούμε να ελέγξουμε αν είναι μάταιο να συνεχίσουμε την αναζήτηση λύσης (γραμμή 6).

Ας ξεκινήσουμε με τα υποψήφια ψηφία που μπορούμε να δοκιμάσουμε για τη λύση σε κάθε θέση. Αυτά είναι τα ψηφία στα οποία μπορούμε να φτάσουμε από τα ψηφία στην ίδια θέση στην αρχική εξίσωση, s_d , βάζοντας μέχρι m_k σπύρτα και βγάζοντας μέχρι m_k σπύρτα.

Όσον αφορά τον έλεγχο για λύση όταν έχουμε φτάσει στην τελική θέση, πρέπει να ελέγξουμε αν ο αριθμός των σπύρτων που θα βάλουμε είναι ίσος με τον αριθμό των σπύρτων που θα βγάλουμε και αν ο αριθμός αυτός είναι το πολύ ίσος με m_k . Αν ισχύει αυτό, θα πρέπει επιπλέον να ελέγξουμε ότι η εξίσωση που προκύπτει είναι αριθμητικά ορθή. Αν είναι, δεχόμαστε τη λύση.

Ας δούμε τώρα πώς μπορούμε να προλάβουμε μάταιες αναζητήσεις. Θα σημειώνουμε το συνολικό αριθμό των σπύρτων που έχουμε προσθέσει με t_a , και τον αριθμό των σπύρτων που έχουμε αφαιρέσει με t_r . Για

κάθε υποψήφιο ψηφίο της λύσης t_d στην τρέχουσα θέση, ενημερώνουμε τα t_a και t_r με τον αριθμό των σπίρτων που χρειάζεται να προσθέσουμε ή να αφαιρέσουμε για να φτάσουμε στο t_d από το s_d . Επίσης δεν πρέπει να ξεχάσουμε το σπίρτο που τυχόν προστέθηκε ή αφαιρέθηκε από τον τελεστή. Συνεπώς, αν $t_a + o_a > m_k$ ή $t_r + o_r > m_k$, δεν μπορεί να προκύψει λύση χρησιμοποιώντας το t_d στη θέση που βρισκόμαστε.

Διαφορετικά, η διαφορά $t_a - t_r$ μας δίνει τον αριθμό των σπίρτων που πρέπει να αφαιρέσουμε (αν είναι θετική) ή να προσθέσουμε (αν είναι αρνητική) για να εξισορροπήσουμε τις μεταφορές σπίρτων στις θέσεις που απομένουν. Μπορούμε επίσης να μεταφέρουμε ένα σπίρτο μεταξύ του τελεστή και των ψηφίων, άρα ο συνολικός αριθμός σπίρτων που πρέπει να προσθέσουμε ή να αφαιρέσουμε είναι $n = o_d - (t_a - t_r)$: αν $n > 0$ πρέπει να προσθέσουμε n σπίρτα, ενώ αν $n < 0$ πρέπει να αφαιρέσουμε n σπίρτα. Ο αριθμός των σπίρτων που μπορούμε να προσθέσουμε είναι $s_a = m_k - o_a - t_a$ ενώ ο αριθμός των σπίρτων που μπορούμε να αφαιρέσουμε είναι $s_r = m_k - o_r - t_r$. Αυτό σημαίνει ότι αν $n > s_a$ ή $n < -s_r$ είναι και πάλι αδύνατον να προκύψει λύση χρησιμοποιώντας το t_d στη θέση που βρισκόμαστε.

Στο σημείο αυτό δεν έχουμε αποκλείσει τη χρήση του t_d στη θέση i αλλά έχουμε ακόμα να εξετάσουμε τις θέσεις $i+1, i+2, \dots, ns-1$. Προκύπτει ότι μπορούμε να προβλέψουμε περιπτώσεις όπου είναι αδύνατον, αν επιλέξουμε το ψηφίο t_d , να βρούμε ψηφία να βάλουμε στις υπόλοιπες θέσεις προσθέτοντας ή αφαιρώντας όχι παραπάνω από n σπίρτα. Ας δούμε βήμα προς βήμα πώς μπορούμε να το προβλέψουμε αυτό.

Έστω ότι έχουμε το ψηφίο $\square$ και μπορούμε να μετακινήσουμε μέχρι δύο σπίρτα. Απαριθμούμε όλα τα υποψήφια ψηφία t_d και υπολογίζουμε τα σπίρτα που πρέπει να προσθέσουμε (αυτά στο S_{t_d} αλλά όχι στο S_0 , δηλαδή a) και να αφαιρέσουμε (αυτά στο S_0 αλλά όχι στο S_{t_d} , δηλαδή r) για να πάμε από το S_d στο S_{t_d} . Για παράδειγμα, μπορούμε να πάμε από το $\square$ στο $\exists$ προσθέτοντας ένα και αφαιρώντας δύο σπίρτα, $\exists$, οπότε $a = 1$ και $r = 2$. Αφού η μετακίνηση ενός σπίρτου είναι μια προσθήκη και μια αφαίρεση, δεν μπορούμε να πάμε από ένα ψηφίο σε ένα άλλο αν $a > m_k = 2$ ή $r > m_k = 2$. Οπότε μπορούμε να απορρίψουμε τα ψηφία στα οποία δεν μπορούμε να πάμε, καταγράφοντας το υπόλοιπο $\delta = a - r$ που πρέπει να καλύψουμε για τα οποία μπορούμε να πάμε. Στην περίπτωσή μας, μπορούμε να πάμε από το $\square$ στο $\exists$ με υπόλοιπο $\delta = a - r = -1$, δηλαδή μπορούμε να αφαιρέσουμε ένα σπίρτο για να φτιάξουμε την υπόλοιπη εξίσωση. Ο παρακάτω πίνακας δείχνει τι συμβαίνει με κάθε t_d αν το τρέχον ψηφίο είναι $\square$. Τα δυνατά υπόλοιπα δ είναι $\{-2, -1, 0, +1\}$, δηλαδή στο διάστημα $[-2, +1]$.

t_d	S_{t_d}	added: $S_{t_d} \setminus S_0$	removed: $S_0 \setminus S_{t_d}$	(a, r)	$\delta = a - r$
$\square$	$\{1, 2, 3, 4, 5, 6\}$	$\emptyset$	$\emptyset$	$(0, 0)$	0
1	$\{2, 3\}$	$\emptyset$	$\{1, 4, 5, 6\}$	$(0, 4)$	$(r > 2)$
2	$\{0, 1, 2, 4, 5\}$	$\{0\}$	$\{3, 6\}$	$(1, 2)$	-1
3	$\{0, 1, 2, 3, 4\}$	$\{0\}$	$\{5, 6\}$	$(1, 2)$	-1
4	$\{0, 2, 3, 6\}$	$\{0\}$	$\{1, 4, 5\}$	$(1, 3)$	$(r > 2)$
5	$\{0, 1, 3, 4, 6\}$	$\{0\}$	$\{2, 5\}$	$(1, 2)$	-1
6	$\{0, 1, 3, 4, 5, 6\}$	$\{0\}$	$\{2\}$	$(1, 1)$	0
7	$\{1, 2, 3, 6\}$	$\emptyset$	$\{4, 5\}$	$(0, 2)$	-2
8	$\{0, 1, 2, 3, 4, 5, 6\}$	$\{0\}$	$\emptyset$	$(1, 0)$	+1
9	$\{0, 1, 2, 3, 4, 6\}$	$\{0\}$	$\{5\}$	$(1, 1)$	0

Τώρα ας υποθέσουμε ότι έχουμε το ψηφίο $\exists$. Με αντίστοιχη απαρίθμηση βρίσκουμε:

t_d	S_{t_d}	added: $S_{t_d} \setminus S_3$	removed: $S_3 \setminus S_{t_d}$	(a, r)	$\delta = a - r$
0	{1, 2, 3, 4, 5, 6}	{5, 6}	{0}	(2, 1)	+1
1	{2, 3}	$\emptyset$	{0, 1, 4}	(0, 3)	$(r > 2)$
2	{0, 1, 2, 4, 5}	{5}	{3}	(1, 1)	0
3	{0, 1, 2, 3, 4}	$\emptyset$	$\emptyset$	(0, 0)	0
4	{0, 2, 3, 6}	{6}	{1, 4}	(1, 2)	-1
5	{0, 1, 3, 4, 6}	{6}	{2}	(1, 1)	0
6	{0, 1, 3, 4, 5, 6}	{5, 6}	{2}	(2, 1)	+1
7	{1, 2, 3, 6}	{6}	{0, 4}	(1, 2)	-1
8	{0, 1, 2, 3, 4, 5, 6}	{5, 6}	$\emptyset$	(2, 0)	+2
9	{0, 1, 2, 3, 4, 6}	{6}	$\emptyset$	(1, 0)	+1

Τα δυνατά υπόλοιπα δ για το 3 είναι $\{-1, 0, +1, +2\}$, δίνοντας το διάστημα $[-1, +2]$.

Για να δούμε ένα ψηφίο όπου απορρίπτουμε επειδή $a > 2$ αντί για $r > 2$, θα πάρουμε το ψηφίο 1. Έχει μόνο δύο σπίρτα, έτσι για να πάμε στα περισσότερα από τα άλλα ψηφία πρέπει να προσθέσουμε πάνω από $m_k = 2$ σπίρτα:

t_d	S_{t_d}	added: $S_{t_d} \setminus S_1$	removed: $S_1 \setminus S_{t_d}$	(a, r)	$\delta = a - r$
0	{1, 2, 3, 4, 5, 6}	{1, 4, 5, 6}	$\emptyset$	(4, 0)	$(a > 2)$
1	{2, 3}	$\emptyset$	$\emptyset$	(0, 0)	0
2	{0, 1, 2, 4, 5}	{0, 1, 4, 5}	{3}	(4, 1)	$(a > 2)$
3	{0, 1, 2, 3, 4}	{0, 1, 4}	$\emptyset$	(3, 0)	$(a > 2)$
4	{0, 2, 3, 6}	{0, 6}	$\emptyset$	(2, 0)	+2
5	{0, 1, 3, 4, 6}	{0, 1, 4, 6}	{2}	(4, 1)	$(a > 2)$
6	{0, 1, 3, 4, 5, 6}	{0, 1, 4, 5, 6}	{2}	(5, 1)	$(a > 2)$
7	{1, 2, 3, 6}	{1, 6}	$\emptyset$	(2, 0)	+2
8	{0, 1, 2, 3, 4, 5, 6}	{0, 1, 4, 5, 6}	$\emptyset$	(5, 0)	$(a > 2)$
9	{0, 1, 2, 3, 4, 6}	{0, 1, 4, 6}	$\emptyset$	(4, 0)	$(a > 2)$

Τα δυνατά υπόλοιπα δ για το 1 είναι $\{0, +2\}$, δίνοντας το διάστημα $[0, +2]$.

Πώς αυτά μας βοηθούν στο να αποφύγουμε μάταιες αναζητήσεις; Ας πούμε ότι έχουμε τον γρίφο $8-9=3$. Έχουμε τρεις θέσεις και μπορούμε να υπολογίσουμε τα διαστήματα των υπολοίπων δ :

slot	digit	$\delta = a - r$
A	8	$[-2, 0]$
B	9	$[-2, +1]$
C	3	$[-1, +2]$

Αν χρησιμοποιούμε τον τελεστή +, πρέπει να προσθέσουμε ένα κάθετο σπίρτο στη θέση G0. Τότε έχουμε $o_a = 1, o_r = 0, o_d = o_r - o_a = 0 - 1 = -1$. Τα ψηφία θα πρέπει συνολικά να προσφέρουν $a - r = -1$ για να εξισορροπήσουν το σπίρτο που μπήκε στο G0. Στην αναζήτησή μας εξετάζουμε το ψηφίο 8 στη θέση A (δηλαδή, αφήνουμε το 8 ως έχει). Δεν αλλάζει τίποτε, οπότε $t_a = t_r = 0$. Συνεχίζουμε στη θέση B. Όταν ελέγχουμε τις εναλλακτικές, θα ελέγξουμε το ψηφίο 8. Αυτό σημαίνει ότι προσθέτουμε το κάθετο σπίρτο B5. Τότε $a = 1, r = 0, t_a = 1, t_r = 0$. Ο συνολικός αριθμός των σπίρτων που πρέπει να προσθέσουμε ή να αφαιρέσουμε είναι $n = o_d - (t_a - t_r) = -2$. Άρα, αν χρησιμοποιήσουμε το ψηφίο 8 στη θέση B, θα πρέπει να προσθέσουμε δύο σπίρτα στη συνέχεια της αναζήτησής μας στη θέση C. Ξέρουμε όμως ότι αφού στη θέση C βρίσκεται το 3, το διάστημα των δ είναι $[-1, +2]$ και το $n = -2$ είναι εκτός αυτού. Ως εκ τούτου, είναι αδύνατον να βρούμε λύση αν υιοθετήσουμε το 8 στη θέση B, χωρίς να χρειαστεί καν να εξετάσουμε εναλλακτικές για τη θέση C!

Γενικά, μπορεί να έχουμε μέχρι έξι θέσεις, όχι μόνο τρεις. Η ίδια λογική όμως ισχύει για κάθε θέση· απλώς

πρέπει να εξετάσουμε τι γίνεται σε όλες τις εναπομείναντες θέσεις. Δηλαδή, αν είμαστε στη θέση C και έχουμε ακόμα τις θέσεις D, E, F, ίσως να μπορούμε να κλαδέψουμε την αναζήτησή μας αμέσως αν τα σπέρτα που πρέπει να προσθέσουμε ή να αφαιρέσουμε δεν μπορούν να καλυφθούν, ό,τι ψηφία και να επιλέξουμε στις θέσεις D, E, F. Αφού οι προσθήκες και οι αφαιρέσεις αθροίζονται πηγαίνοντας από τα αριστερά προς τα δεξιά, αντί να χρησιμοποιούμε απλώς το διάστημα των δ σε κάθε θέση k (δ_k), θα χρησιμοποιούμε τα διαστήματα της κατάληξης (suffix) της εξίσωσης. Ορίζουμε για κάθε $0 \leq i \leq ns$:

$$\text{suf}(i) = [\text{suf}_{\min}(i), \text{suf}_{\max}(i)]$$

με:

$$\text{suf}_{\min}(i) = \sum_{k=i}^{ns-1} \min(\delta_k), \quad \text{suf}_{\max}(i) = \sum_{k=i}^{ns-1} \max(\delta_k),$$

και $\text{suf}_{\min}(ns) = \text{suf}_{\max}(ns) = 0$, καθώς για $i = ns$ έχουμε επεξεργαστεί όλες τις θέσεις $0, 1, \dots, ns-1$: έτσι δείχνουμε ότι στο τέλος δεν μπορούμε να προσθέσουμε ή να αφαιρέσουμε κανένα σπέρτο. Αν βρισκόμαστε στη θέση i , λαμβάνοντας υπόψη και τις θέσεις που απομένουν, αν $n > \text{suf}_{\max}(i+1)$ ή $n < \text{suf}_{\min}(i+1)$ είναι αδύνατον να προκύψει λύση.

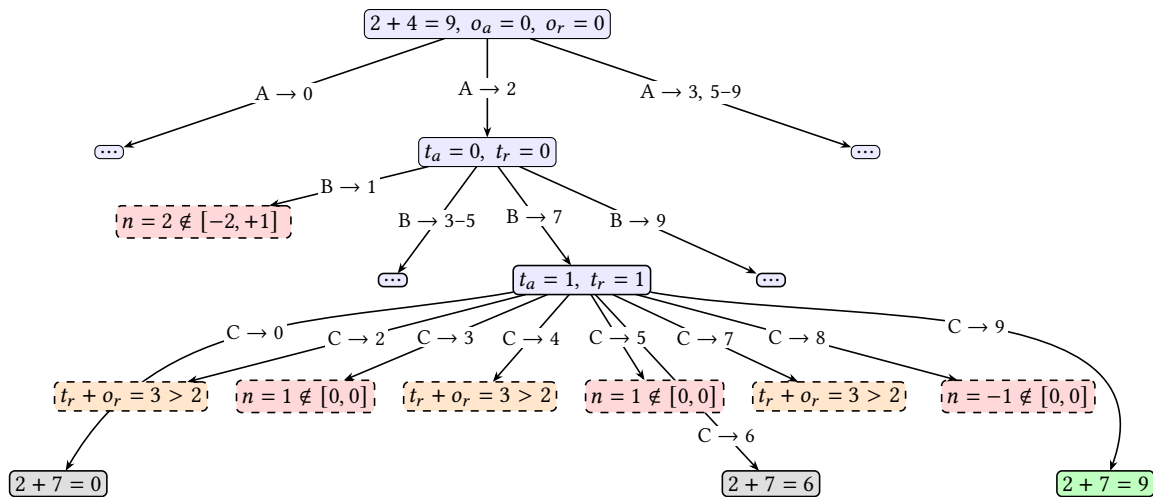
Αν, παρά τις προσπάθειές μας, δεν αποκλείσαμε τη δυνατότητα λύσης, μπορούμε να προχωρήσουμε στην αναζήτηση υιοθετώντας το t_d ως μέρος της λύσης και συνεχίζοντας την εξερεύνηση στην επόμενη θέση.

Η δομή του δένδρου κατά βάθος αναζήτησης και η διαδικασία που περιγράψαμε φαίνονται σχηματικά στην Εικόνα 2 για τον γρίφο $2 + 4 = 9$ έχοντας επιλέξει τον τελεστή $+$. Η εικόνα εστιάζει στο τμήμα του δένδρου αφού επιλέξουμε το ψηφίο 2 για τη θέση A. Κάθε επίπεδο αντιστοιχεί σε μία θέση (A, B, C). Στους κόμβους φαίνονται οι τρέχουσες τιμές (t_a, t_r), και στις ακμές το ψηφίο που επιλέγουμε για την επόμενη θέση. Οι γλαυκοί κόμβοι είναι εσωτερικοί κόμβοι που επισκεπτόμαστε, οι πράσινοι είναι φύλλα που δίνουν λύση, και οι γκριζοί είναι φύλλα που η εξίσωση μαθηματικά δεν ισχύει. Οι διακεκομμένοι κόμβοι είναι κλαδέματα. Τα πορτοκαλί κλαδέματα προέρχονται από τον περιορισμό στον προϋπολογισμό, ενώ τα ροζ κλαδέματα προέρχονται από τον έλεγχο της κατάληξης. Δείτε επίσης την Εικόνα 3 για τον γρίφο $2 + 4 = 9$ έχοντας επιλέξει τον τελεστή $+$, η οποία εστιάζει σε άλλο τμήμα του δένδρου, αυτό που προκύπτει έχοντας επιλέξει το ψηφίο 3 για τη θέση A.

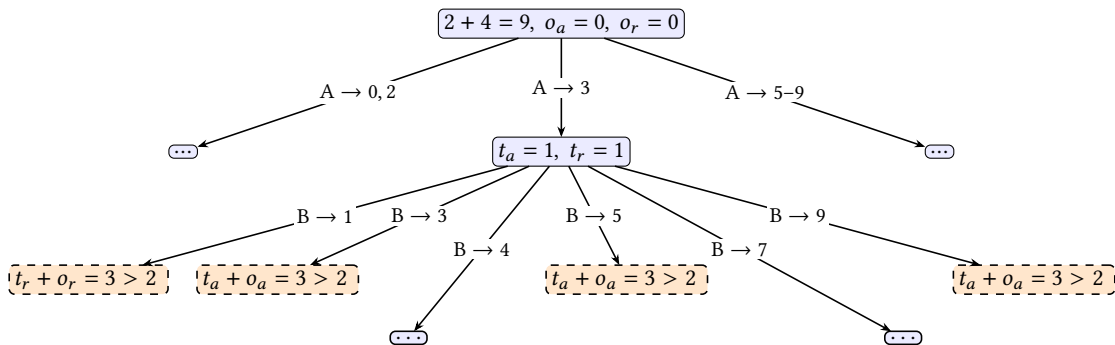
Απαιτήσεις Προγράμματος

Κάθε φοιτητής θα εργαστεί σε αποθετήριο στο GitHub. Για να αξιολογηθεί μια εργασία θα πρέπει να πληροί τις παρακάτω προϋποθέσεις:

- Για την υποβολή της εργασίας θα χρησιμοποιηθεί το ιδιωτικό αποθετήριο του φοιτητή που δημιουργήθηκε για τις ανάγκες του μαθήματος και του έχει αποδοθεί. Το αποθετήριο αυτό έχει όνομα του τύπου `username-algo-assignments`, όπου `username` είναι το όνομα του φοιτητή στο GitHub. Για παράδειγμα, το σχετικό αποθετήριο του διδάσκοντα θα ονομαζόταν `louridas-algo-assignments` και θα ήταν προσβάσιμο στο <https://github.com/dmst-algorithms-course/louridas-algo-assignments>. Τυχόν άλλα αποθετήρια απλώς θα αγνοηθούν.
- Μέσα στο αποθετήριο αυτό θα πρέπει να δημιουργηθεί ένας κατάλογος `assignment-2026-3`.
- Μέσα στον παραπάνω κατάλογο το πρόγραμμα θα πρέπει να αποθηκευτεί με το όνομα `mathstick.py`.



Εικόνα 2: Τμήμα του κατά βάθος δένδρου για τον γρίφο $2 + 4 = 9$ και την επιλογή $A \rightarrow 2$.



Εικόνα 3: Τμήμα του κατά βάθος δένδρου για τον γρίφο $2 + 4 = 9$ και την επιλογή $A \rightarrow 3$.

- Δεν επιτρέπεται η χρήση έτοιμων βιβλιοθηκών ή τυχόν έτοιμων υλοποιήσεων των αλγορίθμων, ή τμημάτων αυτών, εκτός αν αναφέρεται ρητά ότι επιτρέπεται.
- Επιτρέπεται η χρήση δομών δεδομένων της Python όπως στοίβες, λεξικά, σύνολα, κ.λπ.
- Επιτρέπεται η χρήση των παρακάτω βιβλιοθηκών ή τμημάτων τους όπως ορίζεται:
 - sys
 - argparse
 - json
 - re
- Το πρόγραμμα θα πρέπει να είναι γραμμένο σε Python 3.
- Η εργασία είναι αποκλειστικά ατομική. Δεν επιτρέπεται συνεργασία μεταξύ φοιτητών στην εκπόνησή της, με ποινή το μηδενισμό. Επιπλέον η εργασία δεν μπορεί να είναι αποτέλεσμα συστημάτων Τεχνητής Νοημοσύνης. Ειδικότερα όσον αφορά το τελευταίο σημείο προσέξτε ότι τα συστήματα αυτά χωλαίνουν στην αλγοριθμική σχέση, άρα τυχόν προτάσεις που κάνουν σε σχετικά θέματα μπορεί να είναι λανθασμένες. Επιπλέον, αν θέλετε να χρησιμοποιήσετε τέτοια συστήματα, τότε αν και κάποιος άλλος το κάνει αυτό, μπορεί οι προτάσεις που θα λάβετε να είναι παρόμοιες, οπότε οι

εργασίες θα παρουσιάσουν ομοιότητες και άρα θα μηδενιστούν.

- Η έξοδος του προγράμματος θα πρέπει να περιλαμβάνει μόνο ό,τι φαίνεται στα παραδείγματα που παρατίθενται.

Η φλυαρία δεν επιβραβεύεται.

Χρήση του GitHub

Όπως αναφέρθηκε το πρόγραμμά σας για να αξιολογηθεί θα πρέπει να αποθηκευθεί στο GitHub. Επιπλέον, θα πρέπει το GitHub να χρησιμοποιηθεί καθ' όλη τη διάρκεια της ανάπτυξης του. Θα εργαστείτε στο αποθετήριο που σας έχει αποδοθεί και:

- δεν θα κάνετε fork κάποιο άλλο αποθετήριο (όπως της εκφώνησης)
- θα εργαστείτε στο main branch
- δεν θα κάνετε pull και merge requests

Προσέξτε ότι *σίγουρα δεν θα ανεβάσετε στο GitHub απλώς την τελική λύση της εργασίας*. Στο GitHub θα πρέπει να φαίνεται το ιστορικό της συγγραφής του προγράμματος. Αυτό συνάδει και με τη φιλοσοφία του εργαλείου: λέμε “commit early, commit often”. Εργαζόμαστε σε ένα πρόγραμμα και κάθε μέρα, ή όποια στιγμή έχουμε κάνει κάποιο σημαντικό βήμα, αποθηκεύουμε την αλλαγή στο GitHub. Αυτό έχει σειρά ευεργετικών αποτελεσμάτων:

- Έχουμε πάντα ένα αξιόπιστο εφεδρικό μέσο στο οποίο μπορούμε να ανατρέξουμε αν κάτι πάει στραβά στον υπολογιστή μας (μας γλιτώνει από πανικούς του τύπου: ένα βράδυ πριν από την παράδοση ο υπολογιστής μας ή ο δίσκος του πνέει τα λοίσθια, και εμείς τι θα υποβάλουμε στον Λουρίδα;).
- Καθώς έχουμε πλήρες ιστορικό των σημαντικών αλλαγών και εκδόσεων του προγράμματός μας, μπορούμε να επιστρέψουμε σε μία προηγούμενη αν συνειδητοποιήσουμε κάποια στιγμή ότι πήραμε λάθος δρόμο (μας γλιτώνει από πανικούς του τύπου: μα το πρωί δούλεψε σωστά, τι στο καλό έκανα και τώρα δεν παίζει τίποτε;).
- Καθώς φαίνεται η πρόοδος μας στο GitHub, επιβεβαιώνουμε στον διδάσκοντα ότι πραγματικά έχουμε εργαστεί μεθοδικά και δεν μας δώθηκε η λύση στο πιάτο (μας γλιτώνει από πανικούς του τύπου: καλά, πώς το έλυσες το πρόβλημα αυτό με τη μία, μόνος σου;).
- Αν δουλεύετε σε μία ομάδα που βεβαίως δεν είναι καθόλου η περίπτωση μας εδώ, μπορούν όλοι να εργάζονται στα ίδια αρχεία στο GitHub εξασφαλίζοντας ότι ο ένας δεν γράφει πάνω στις αλλαγές του άλλου. Παρά το ότι η εργασία αυτή είναι ατομική, καλό είναι να αποκτάτε τριβή με το εργαλείο git και την υπηρεσία GitHub μιας και χρησιμοποιούνται ευρέως και όχι μόνο στη συγγραφή κώδικα.

Άρα επενδύστε λίγο χρόνο στην εκμάθηση των git / GitHub, των οποίων ο σωστός τρόπος χρήσης είναι μέσω γραμμής εντολών (command line), ή ειδικών προγραμμάτων (clients) ή μέσω ενοποίησης στο περιβάλλον ανάπτυξης (editor, IDE).

Όσον αφορά την αξιολόγηση της εργασίας, θα ληφθεί υπόψη ακριβώς η ιστορικότητα που θα έχει καταχωρηθεί στο GitHub. Δεν θα αξιολογηθούν εργασίες οι οποίες στο GitHub εμφανίζουν μόνο ένα commit,

ή ελάχιστα commit. Η ιστορικότητα της εργασίας, όπως αποτυπώνεται στο GitHub, θα πρέπει να αντανακλά την πρόοδο στην εκπόνησή της από την αρχή της υλοποίησης μέχρι το τέλος της προθεσμίας.

Εργασίες που υποβάλλονται ως επιφώτηση δεν θα βαθμολογούνται.

Χρήση Προγράμματος

Το πρόγραμμα θα καλείται ως εξής (όπου python η κατάλληλη εντολή στο εκάστοτε σύστημα):

```
python mathstick.py --problem PROBLEM [--max-k MAX_K]
```

Το πρόγραμμα παίρνει ως είσοδο ένα πρόβλημα, σε μορφή συμβολοσειράς. Η παράμετρος --max-k δίνει το m_k . αν δεν δοθεί, θα είναι ίση με 2.

Στην έξοδο του προγράμματος θα εμφανιστεί, σε μορφή JSON, το αποτέλεσμα του (δείτε παρακάτω στα παραδείγματα). Οι λύσεις, εφόσον υπάρχουν, παρατίθενται με τη σειρά που βρέθηκαν στην κατά βάθος αναζήτηση.

Για κάθε λύση, όπως θα δείτε, περιλαμβάνεται ο αριθμός των κόμβων του δένδρου της κατά βάθος αναζήτησης που επισκέφτηκε το πρόγραμμα, και ο αριθμός των κόμβων που έχουμε κλαδέψει. Επίσης για κάθε πρόβλημα δίνεται και ο συνολικός αριθμός κόμβων που επισκέφτηκε και που κλάδεψε. Προσέξτε ότι τα σύνολα δεν είναι τα επιμέρους αθροίσματα, καθώς δεν οδηγούν όλες οι κατά βάθος αναζητήσεις σε λύσεις. Για παράδειγμα, για το πρόβλημα $1 + 4 = 9$, βρίσκουμε μια λύση όταν έχουμε επισκεφτεί 51 κόμβους και έχουμε κλαδέψει 125 κόμβους· η κατά βάθος αναζήτηση συνεχίζει για τυχόν άλλες λύσεις, και τερματίζει αφού επισκεφτεί 52 κόμβους και κλαδέψει 136.

Η πλήρης μορφοποίηση των αποτελεσμάτων του προγράμματος περιγράφεται μέσω JSON Schema στο αρχείο mathstick_solver_output_schema.json.

Παραδείγματα

Παράδειγμα 1

Αν ο χρήστης δώσει:

```
python mathstick.py --problem '1 + 4 = 9'
```

το πρόγραμμα θα παράξει την παρακάτω έξοδο:

```
{
  "problem": "1 + 4 = 9",
  "max_k": 2,
  "counts": {
    "1": 0,
    "2": 1
  },
  "nodes_visited": 52,
  "nodes_pruned": 136,
  "solutions": {
```



```

"1": [],
"2": [
  {
    "equation": "7 - 4 = 3",
    "picks": [
      "C6",
      "G0"
    ],
    "places": [
      "A1",
      "A6"
    ],
    "moves": [
      "Move(C6, A1)",
      "Move(G0, A6)"
    ],
    "nodes_visited": 51,
    "nodes_pruned": 125
  }
]
}
}

```

Παράδειγμα 2

Αν ο χρήστης δώσει:

```
python mathstick.py --problem '2 + 4 = 9'
```

το πρόγραμμα θα παράξει την παρακάτω έξοδο:

```

{
  "problem": "2 + 4 = 9",
  "max_k": 2,
  "counts": {
    "1": 2,
    "2": 3
  },
  "nodes_visited": 76,
  "nodes_pruned": 225,
  "solutions": {
    "1": [
      {
        "equation": "2 + 4 = 6",
        "picks": [
          "C2"
        ],
        "places": [
          "C5"
        ],

```

```

    "moves": [
      "Move(C2, C5)"
    ],
    "nodes_visited": 6,
    "nodes_pruned": 4
  },
  {
    "equation": "2 + 7 = 9",
    "picks": [
      "B0"
    ],
    "places": [
      "B1"
    ],
    "moves": [
      "Move(B0, B1)"
    ],
    "nodes_visited": 8,
    "nodes_pruned": 7
  }
],
"2": [
  {
    "equation": "2 + 3 = 5",
    "picks": [
      "B6",
      "C2"
    ],
    "places": [
      "B1",
      "B4"
    ],
    "moves": [
      "Move(B6, B1)",
      "Move(C2, B4)"
    ],
    "nodes_visited": 16,
    "nodes_pruned": 21
  },
  {
    "equation": "8 + 1 = 9",
    "picks": [
      "B0",
      "B6"
    ],
    "places": [
      "A3",
      "A6"
    ],

```

```

    ],
    "moves": [
        "Move(B0, A3)",
        "Move(B6, A6)"
    ],
    "nodes_visited": 32,
    "nodes_pruned": 59
},
{
    "equation": "5 + 4 = 9",
    "picks": [
        "A2",
        "A5"
    ],
    "places": [
        "A3",
        "A6"
    ],
    "moves": [
        "Move(A2, A3)",
        "Move(A5, A6)"
    ],
    "nodes_visited": 47,
    "nodes_pruned": 107
}
]
}
}

```

Παράδειγμα 3

Αν ο χρήστης δώσει:

```
python mathstick.py --problem '1 + 1 = 1'
```

το πρόγραμμα θα παράξει την παρακάτω έξοδο:

```

{
    "problem": "1 + 1 = 1",
    "max_k": 2,
    "counts": {
        "1": 0,
        "2": 0
    },
    "nodes_visited": 7,
    "nodes_pruned": 13,
    "solutions": {
        "1": [],
        "2": []
    }
}

```

}

Επιπλέον Παραδείγματα

Στο αρχείο `mathsticks_bench_jsonl.zip` θα βρείτε όλα τα προβλήματα με τις λύσεις τους για διαφορετικό αριθμό ψηφίων. Μπορείτε να τα χρησιμοποιήσετε για να ελέγξετε τις λύσεις σας.

Καλή Επιτυχία!